Project Management for Lands, Constructions etc.

Technical and Functional Documentation — Frappe/ERPNext v16 custom application `project_management`, installed on site `asset.localhost`.

Item	Value
App name	`project_management`
App title	Project Management for lands constructions etc
Publisher	Prajwal k
License	MIT
Framework	Frappe v16 (bench app, Python controllers + Desk client scripts)
Modules	Project Masters; Project Management for lands constructions etc
External dependency	`frappe_whatsapp` (WhatsApp Message / WhatsApp Account)
Document count	11 DocTypes (8 standard, 1 single, 3 child tables)
Document date	15 October 2026

Contents

Section	Subject
1. Purpose and scope	What the app does and what this document covers
2. System overview	Masters vs. execution layers, data model map, naming series
3. Roles and authorisation model	Roles, row-level permission hooks, user-to-employee mapping
4. Functional walkthrough	End-to-end business flow, configuration to completion
5. DocType reference	5.1 Task Master · 5.2 Type of Work Masters · 5.3 Project Master Template · 5.4 Project Task Detail · 5.5 Project Execution · 5.6 Project Task Instance · 5.7 Task Remarks History · 5.8 Project Tracker · 5.9 WhatsApp Reminder Log · 5.10 WhatsApp Status Recipient · 5.11 Project Management Settings
6. Reminder engine	Scheduling, dispatch re-checks, interval arithmetic
7. WhatsApp messaging	The three outbound kinds and link rewriting
8. Whitelisted API surface	Every server endpoint and its guard
9. Hooks, migrations and installation	hooks.py, the five patches, install commands
10. Data lifecycle and deletion rules	What each action creates, updates or removes
11. Configuration checklist	Ordered setup steps for a new installation
12. Troubleshooting	Error messages and their causes
13. Source map	Where each behaviour lives in the repository

1. Purpose and scope

The application turns repeatable project playbooks into tracked, reminder-driven execution for an owner running several lines of business (land, construction, trading). An administrator configures reusable masters once; an employee then only picks a project template, names the project, selects a task, types remarks and chooses a status.

This document covers, for every DocType in the app:

- what it is for in business terms (functional view);
- every field with type, target DocType, default and behavioural flags (technical view);
- server-side controller logic, validations and derived values;
- the role permission matrix as shipped;
- how it connects to the rest of the data model.

It also documents the reminder scheduler, the WhatsApp delivery layer, the whitelisted API surface, hooks, migration patches and operational configuration.

2. System overview

2.1 Two layers: masters and live execution

The data model separates **configuration** from **execution**. Project Masters holds reusable definitions; the execution module holds one independent copy per live project, so two projects built from the same template never share status, assignees, due dates, remarks or reminder state.

Layer	DocTypes	Who maintains it
Masters (configuration)	Task Master, Type of Work Masters, Project Master Template, Project Task Detail	Admin Master / System Manager
Execution (live data)	Project Execution, Project Task Instance, Task Remarks History, Project Tracker	Created by the system; updated by Concern Persons
Messaging and configuration	WhatsApp Reminder Log, WhatsApp Status Recipient, Project Management Settings	Scheduler writes logs; settings are administrator-owned

2.2 Data model map

```
Type of Work Masters ──┐
                       ├─> Project Master Template (submittable)
Task Master ───────────┘          |  task_details (child: Project Task Detail)
     |                            |
     |                            v
     |                 Project Execution  (PM-PRJ-#####)   live project
     |                            |  after_insert: one task instance per template row
     |                            v
     └──────────────────> Project Task Instance (PM-TASK-#####)
                                  |  remarks_history (child: Task Remarks History)
                                  ^
                                  |  appends one row per save
                       Project Tracker  (PM-TRK-####)   employee entry screen
                                  |  owns exactly one Project Execution (unique)
                                  v
                       WhatsApp Reminder Log (PM-WA-#####)  every outbound attempt
                       Project Management Settings (single)  switches + recipients
```

2.3 Naming series

DocType	Series / naming rule
Task Master	`TASK-.####`
Type of Work Masters	`WORK-.####`
Project Master Template	named by `project_name` (unique business name)
Project Execution	`PM-PRJ-.#####`
Project Task Instance	`PM-TASK-.#####`
Project Tracker	`PM-TRK-.####`
WhatsApp Reminder Log	`PM-WA-.#####`
Project Management Settings	single document, no series
Project Task Detail / Task Remarks History / WhatsApp Status Recipient	child rows, hashed names

3. Roles and authorisation model

Three roles are relevant. `Admin Master` and `Concern Person` are created by the app's `after_install` / `after_migrate` hook (`project_management.install.after_install`); `System Manager` is a framework role.

Role	Intent	Effective reach
System Manager	Technical administrator	Everything, including deleting projects and logs
Admin Master	Business owner / project office	All masters, all projects, all tasks; may send WhatsApp status and manual reminders
Concern Person	Employee responsible for tasks	Reads masters; sees and updates only task instances assigned to them; may create a tracker entry

Row-level filtering is enforced on the server in `project_management/permissions.py`, wired through `hooks.py`:

Hook	DocType	Function
`permission_query_conditions`	Project Task Instance	`task_instance_query_conditions` — appends `concern_person = <employee>`; returns `1=0` when the user has no active Employee record
`permission_query_conditions`	Project Tracker	`tracker_query_conditions` — own entries only
`has_permission`	Project Task Instance	`has_task_instance_permission` — document-level equivalent of the query condition
`has_permission`	Project Tracker	`has_tracker_permission` — allows an unsaved document (the controller fills `concern_person` during validate), then restricts to own entries

> Button visibility is never treated as authorisation. The two restricted actions (`send_status_to_admin_master`, `send_manual_task_reminder`) call `require_reminder_permission()` server-side, and `Project Tracker.validate_assignment()` re-checks task ownership on every save.

A user is mapped to an Employee by `get_employee_for_user(user)`: `Employee` where `user_id = <session user>` and `status = Active`. `Administrator` is treated as elevated and maps to no Employee. Elevated roles are `('Admin Master', 'System Manager', 'Administrator')`.

4. Functional walkthrough

1. **Define vocabulary.** Create `Task Master` records (the reusable task names) and `Type of Work Masters` records (Land, Construction, Trading…).
2. **Build a playbook.** Create a `Project Master Template`: pick the type of work and add one task row per stage — task, concern person, completion timeline in days, reminders on/off, reminder interval in days. **Submit** it; only submitted, non-disabled templates can produce live projects.
3. **Start a project.** The employee opens `Project Tracker`, picks the template, types the project name and optionally a start date. The template's task rows render as a read-only pipeline preview before the first save.
4. **Report progress.** In the Entry Form, choose the task, type remarks, set WIP or Completed, optionally override the overall Project Status, and save.
5. **First save creates the live project.** The tracker creates its own `Project Execution` plus one `Project Task Instance` per template row, each with

`due_date = start_date + completion_timeline_in_days` and the first reminder scheduled; then it writes the remark to the selected task. Template, project name and start date become read-only afterwards.

6. **Every later save appends.** The remark is added to the selected task's `Task Remarks History` and never overwrites an earlier one. To report the next task of the same project, change **Task** on the same tracker entry and save again — the previous task keeps everything written on it. Re-saving with an unchanged remark and status adds nothing.
7. **Completion.** Setting a task to Completed stamps `completed_on`, clears the reminder schedule, and refreshes the project's counters and progress percentage.
8. **Reminders.** While a task stays WIP, the hourly scheduler sends a WhatsApp reminder to the assigned Concern Person every `reminder_in_days` days, counted from the latest remark or the last successful send.
9. **Oversight.** Admin Master can send a project status summary to configured recipients, or a manual reminder with an extra note, from the tracker or the project.

5. DocType reference

Sections 5.1 to 5.11 follow the lifecycle order: masters, template, live project, task, history, tracker, log, settings. Field tables are generated from the DocType JSON; the `Properties` column lists the framework flags actually set on the field.

5.1 Task Master — Project Masters

Functional role. The controlled vocabulary of task names used everywhere else: template rows, live task instances and remark history all point at a Task Master record. Keeping task names here means a renamed or retired task is corrected in one place instead of in every project.

- Module: **Project Masters**
- Naming: naming series `TASK-.####`
- Title field: `task_master`
- Rename allowed: yes
- Change tracking (audit trail of field edits): on

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
`naming_series`	Task ID	Select	TASK-.####	TASK-.####	Mandatory, Set once	
`task_master`	Task Master	Data			Mandatory, Unique, List view, Standard filter	
`disabled`	Disabled	Check		0		
`description`	Description	Small Text				

Behaviour. `task_master` (the display name) is mandatory and unique, so the same task cannot be created twice; `disabled` retires a task from new configuration without touching history. `description` is free text explaining what the task covers. Controller `task_master.py` adds no validation beyond the schema.

Permissions

Role	Granted
System Manager	Read, Write, Create, Delete, Report, Export, Print, Email, Share
Admin Master	Read, Write, Create, Report, Export, Print, Email, Share
Concern Person	Read

5.2 Type of Work Masters — Project Masters

Functional role. Business-line classification for templates and projects — Land, Construction, Trading, and anything else the owner runs. It is the reporting dimension that lets one installation serve several businesses.

- Module: **Project Masters**
- Naming: naming series `WORK-.####`
- Title field: `work_name`
- Rename allowed: yes
- Change tracking (audit trail of field edits): on

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
`naming_series`	Work ID	Select	WORK-.####	WORK-.####	Mandatory, Set once	
`work_name`	Work Name	Data			Mandatory, Unique, List view, Standard filter	
`disabled`	Disabled	Check		0		

Behaviour. `work_name` is mandatory, unique and a standard filter, so list views can be sliced by business line. `Project Execution.type_of_work` is fetched from the template, which means the classification follows the project automatically and cannot be set inconsistently.

Permissions

Role	Granted
System Manager	Read, Write, Create, Delete, Report, Export, Print, Email, Share
Admin Master	Read, Write, Create, Report, Export, Print, Email, Share
Concern Person	Read

5.3 Project Master Template — Project Masters

Functional role. The reusable playbook: one business line plus the ordered list of tasks with who owns each, how long it should take and how often to chase it. It is submittable, because a template that live projects are built from must be frozen and auditable, not edited underneath them.

- Module: **Project Masters**
- Submittable: draft (0) → submitted (1) → cancelled (2)
- Naming: value of `project_name` (document name = that value)
- Title field: `project_name`
- Rename allowed: yes
- Change tracking (audit trail of field edits): on

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
`project_name`	Project Name	Data			Mandatory, Unique, List view	
`type_of_work`	Type of Work	Link	Type of Work Masters		Mandatory, List view, Standard filter	
`disabled`	Disabled	Check		0		
`amended_from`	Amended From	Link	Project Master Template		Read-only	
Task Details						
`task_details`	Task Details	Table	Project Task Detail		Mandatory	

Validation (`validate_tasks`, runs on every save and before submit)

- At least one row in Task Details — an empty playbook would create a project with no tasks.
- No duplicate `task` across rows: each task may appear only once per template.
- `completion_timeline_in_days` must be a positive integer on every row — it is the input to every due date.
- When `enable_reminders` is set, `reminder_in_days` must be a positive integer.
- When `enable_reminders` is cleared, `reminder_in_days` is forced to 0, so a stale interval cannot survive turning reminders off.

Actions

- **Create Live Project** (client button, visible only when `docstatus = 1` and not disabled) calls the whitelisted `create_project_execution(project_title, start_date)`, which re-checks submission server-side, creates a `Project Execution` carrying the template, title, type of work and start date, and routes the user to it. A project created this way has no owning tracker.
- Child-row client logic: toggling **Reminders** in a row sets `reminder_in_days` to 0 when switched off and defaults it to 7 when switched on.

Permissions

Role	Granted
System Manager	Read, Write, Create, Delete, Submit, Cancel, Amend, Report, Export, Print, Email, Share
Admin Master	Read, Write, Create, Submit, Cancel, Amend, Report, Export, Print, Email, Share
Concern Person	Read

5.4 Project Task Detail — Project Masters (child table)

Functional role. One planned stage inside a template: which task, who is responsible, how many calendar days it should take, and the reminder cadence. These rows are the blueprint that `Project Execution` copies into live task instances.

- Module: **Project Masters**
- Child table: rows exist only inside a parent document
- Rename allowed: yes
- Change tracking (audit trail of field edits): off

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
`task`	Task	Link	Task Master		Mandatory, List view	
`task_name`	Task Name	Data			Read-only	Fetched from `task.task_master`.
`concern_person`	Concern Person	Link	Employee		Mandatory, List view	
`employee_name`	Employee Name	Data			Read-only	Fetched from `concern_person.employee_name`.
`completion_timeline_in_days`	Completion Timeline in Days	Int		1	Mandatory, List view	
`reminder_in_days`	Reminder in Days	Int		7	List view	Shown when `enable_reminders`.
`enable_reminders`	Reminders	Check		1	List view	

Behaviour. Row order (`idx`) is meaningful: it becomes `Project Task Instance.sequence` and therefore the display order of the pipeline. `task_name` and `employee_name` are fetched read-only copies for grid readability. `reminder_in_days` is hidden unless `enable_reminders` is on. The row is validated by the parent template, not by its own controller.

Permissions

No permission rules of its own: access is inherited from the parent document.

5.5 Project Execution — live project

Functional role. A real project under way: the site, plot or deal being executed. It is the aggregate record — progress counters, expected end date and overall status — and the parent of the live task instances. It stays a first-class DocType: it can be opened directly to review progress, and it can be created from a submitted template without any tracker.

- Module: **Project Management for lands constructions etc**
- Naming: naming series `PM-PRJ-.#####`
- Title field: `project_title`
- Rename allowed: no
- Change tracking (audit trail of field edits): on
- Dashboard link: **Project Task Instance** via `project_execution`

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
`naming_series`	Series	Select	PM-PRJ-.#####	PM-PRJ-.#####	Set once	
`project_title`	Project Title	Data			Mandatory, List view	
`project_master`	Project Master Template	Link	Project Master Template		Mandatory, Set once, List view	
`type_of_work`	Type of Work	Link	Type of Work Masters		Read-only, Standard filter	Fetched from `project_master.type_of_work`.
`status`	Status	Select	Active / Completed / On Hold / Cancelled	Active	Mandatory, List view, Standard filter	
`start_date`	Activation / Start Date	Date			Mandatory, Set once	
`expected_end_date`	Expected End Date	Date			Read-only	
Progress						
`total_tasks`	Total Tasks	Int			Read-only	
`completed_tasks`	Completed	Int			Read-only	
`wip_tasks`	WIP	Int			Read-only	
`progress_percent`	Progress	Percent			Read-only	
Task Pipeline						
`tasks_html`	Tasks	HTML				

Controller logic (`project_execution.py`)

Hook / method	What it does
`validate`	Defaults `start_date` to today when left empty.
`after_insert` → `create_task_instances`	Loads the template, refuses a template that is not submitted, then inserts one `Project Task Instance` per `task_details` row — copying task, task name, `sequence = row.idx`, concern person, completion timeline, reminder switch and reminder interval — with `current_status = WIP` and `activation_date = project start date`.
`after_insert` → `update_progress`	Computes the counters immediately so a new project shows its task count.
`update_progress(status=None)`	Re-reads all task instances and writes `total_tasks`, `completed_tasks`, `wip_tasks = total − completed`, `progress_percent` (2 decimals), `expected_end_date = max(task due dates)` and the derived `status` in a single `db_set` with `update_modified=False`, so refreshing counters does not look like a user edit.
`derive_status(total, completed, status)`	An explicit status always wins. Otherwise: a Cancelled project is never revived by progress; all tasks completed promotes to Completed; a manual On Hold is not demoted by partial progress; anything else is Active.
`get_pipeline()` (whitelisted)	Returns the ordered stage list with status and full remark history for the form's HTML pipeline.

Derived vs. entered. Only `project_title`, `project_master`, `status` and `start_date` are entered. `type_of_work` is fetched from the template; `expected_end_date`, the four progress fields and the task pipeline are computed. `tasks_html` is a rendering target, not stored data.

Permissions

Role	Granted
System Manager	Read, Write, Create, Delete, Report, Export, Print, Email, Share
Admin Master	Read, Write, Create, Delete, Report, Export, Print, Email, Share
Concern Person	Read, Report

> A Concern Person has read/report only on Project Execution. Project creation during a tracker save therefore runs with `ignore_permissions=True` on a fully constrained payload — otherwise an employee recording the first update of a project would be blocked by the project's own permissions.

5.6 Project Task Instance — the live task

Functional role. The unit of work people actually report on: one task of one project, with its own assignee, due date, status, reminder schedule and remark history. It is the reason two projects from the same template are fully independent — each has its own instances.

- Module: **Project Management for lands constructions etc**
- Naming: naming series `PM-TASK-.#####`
- Title field: `task_name`

- Rename allowed: no
- Change tracking (audit trail of field edits): on

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
naming_series	Series	Select	PM-TASK-.#####	PM-TASK-.#####	Set once	
project_execution	Project	Link	Project Execution		Mandatory, Read-only, List view, Standard filter	
project_title	Project Title	Data			Read-only	Fetched from project_execution.project_title.
task	Task	Link	Task Master		Mandatory, Read-only, List view	
task_name	Task Name	Data			Read-only, List view	
sequence	Sequence	Int			Read-only	
concern_person	Concern Person	Link	Employee		Mandatory, List view, Standard filter	
concern_person_name	Concern Person Name	Data			Read-only	Fetched from concern_person.employee_name.
concern_person_mobile	Concern Person Mobile	Data	Phone			Fetched from concern_person.cell_number. Defaults to the Concern Person's Mobile; set a number here to override it for this task.
current_status	Current Status	Select	WIP / Completed	WIP	Mandatory, List view, Standard filter	
Schedule & Reminders						
activation_date	Activation / Start Date	Date			Read-only	
completion_timeline_in_days	Completion Timeline in Days	Int			Read-only	
due_date	Due Date	Date			Read-only, List view	
completed_on	Completed On	Datetime			Read-only	
enable_reminders	Reminders Enabled	Check		1		
reminder_in_days	Reminder in Days	Int				Shown when enable_reminders.
last_remark_on	Last Remark On	Datetime			Read-only	
next_reminder_due	Next Reminder Due	Datetime			Read-only	
last_reminder_sent_on	Last Successful Reminder On	Datetime			Read-only	
reminder_state	Reminder State	Select	Scheduled / Sent / Skipped / Failed / Disabled	Scheduled	Read-only	
Remarks History						
remarks_history	Remarks History	Table	Task Remarks History		Read-only	

Controller logic (`project_task_instance.py`)

Method	What it does
validate	Runs set_due_date then sync_reminder_schedule.
set_due_date	due_date = activation_date + completion_timeline_in_days in calendar days, when both are present.
sync_reminder_schedule	Completed task: clears next_reminder_due, sets reminder_state = Disabled, stamps completed_on if unset. Re-opened task: clears completed_on. Reminders off or interval ≤ 0: clears the due slot and sets Disabled. Otherwise recomputes next_reminder_due and lifts the state from blank/Disabled to Scheduled.
reminder_anchor	The event the next reminder counts from: the later of last_reminder_sent_on and last_remark_on, falling back to the record's creation time.
compute_next_reminder_due(anchor)	anchor + reminder_in_days, snapped to midnight. Day granularity is deliberate: the dispatch job runs in one fixed hour, so a due slot carrying the anchor's clock time would be missed on its own day and the interval would drift by a day every cycle.

`sync_tracker_remark(...)`	Appends exactly one `Task Remarks History` row per new remark — task, remark text, status at entry, previous status, `status_changed`, entry time, concern person and name, entering user and the tracker entry it came from — then recomputes and saves. A save repeating the most recent remark *and* status returns without appending, so re-saving an unchanged form cannot pad the history. Empty remarks are rejected.
`latest_remark` / `sorted_history`	History is ordered by `entry_datetime`, then grid `idx` as tiebreaker; the latest row drives current state.
`recompute_from_history`	Derives `current_status` and `last_remark_on` from the newest history row (falling back to WIP with no remarks), so editing or deleting tracker entries cannot leave a stale status. A new remark on a WIP task resets `reminder_state` to `Scheduled`.
`mark_reminder_sent(sent_on)`	Stamps `last_reminder_sent_on`, sets `reminder_state = Sent` and recomputes `next_reminder_due` from the later of the send and the last remark — all with `update_modified=False`.
`remove_tracker_remark(tracker_entry)`	Drops history rows written by one tracker entry and reindexes. Used by migration patches only; normal saves append the next remark instead of retracting one.

Mobile number resolution. `concern_person_mobile` is fetched from the Employee's `cell_number` but remains editable, so a task can be chased on a different number without changing the Employee master. It is the number every automatic and manual reminder uses.

Permissions

Role	Granted
System Manager	Read, Write, Create, Delete, Report, Export, Print, Email, Share
Admin Master	Read, Write, Create, Report, Export, Print, Email, Share
Concern Person	Read, Write, Report

> Admin Master has no `delete` on task instances; deletion happens through `Project Tracker.on_trash` (with `ignore_permissions`) or by a System Manager. Concern Person has read/write/report, narrowed to their own rows by the permission hooks.

5.7 Task Remarks History — append-only audit trail (child table)

Functional role. The progress narrative of a task: every remark anyone wrote, when, under which status, and whether that entry changed the status. It is the evidence behind the pipeline view and the content of every reminder message.

- Module: **Project Management for lands constructions etc**
- Child table: rows exist only inside a parent document
- Rename allowed: yes
- Change tracking (audit trail of field edits): off

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
`task`	Task	Link	Task Master		Read-only, List view	
`task_name`	Task Name	Data			Read-only	
`remarks`	Remarks	Text			Read-only, List view	
`status_at_entry`	Status	Select	WIP / Completed		Read-only, List view	
`previous_status`	Previous Status	Select	(blank) / WIP / Completed		Read-only	
`status_changed`	Status Changed	Check		0	Read-only	Set when this entry changed the task status.
`entry_datetime`	Date & Time	Datetime			Read-only, List view	
`concern_person`	Concern Person	Link	Employee		Read-only, List view	
`concern_person_name`	Concern Person Name	Data			Read-only	
`entered_by`	Entered By	Link	User		Read-only	
`tracker_entry`	Tracker Entry	Link	Project Tracker		Read-only	

Behaviour

- Every field is read-only in the UI: rows are written by `sync_tracker_remark`, never typed into the grid.
- `previous_status` and `status_changed` make a status transition visible without diffing rows.
- `tracker_entry` records which Project Tracker entry produced the row — the link used by migration patches and audit review.
- `sort_order` is DESC on creation, so the newest remark is shown first in the grid; the pipeline renderer sorts ascending by `entry_datetime` and shows the full trail.
- `entry_datetime` is stamped when the remark is written, not when the tracker entry was created, because reminders compare against it.

Permissions

No permission rules of its own: access is inherited from the parent document.

5.8 Project Tracker — the employee entry screen

Functional role. The single screen an employee uses. It answers four questions — which project, which task, what happened, what is the status — and does everything else implicitly: creating the live project, resolving the task instance, appending the remark and refreshing project progress. One tracker owns exactly one project (`project` is unique).

- Module: **Project Management for lands constructions etc**
- Naming: naming series `PM-TRK-.####`
- Title field: `project_name`

- Rename allowed: no
- Change tracking (audit trail of field edits): on

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
Project						
`naming_series`	Tracker ID	Select	PM-TRK-.####	PM-TRK-.####	Set once	
`project_template`	Project Template	Link	Project Master Template		Mandatory, Set once, List view, Standard filter	Submitted Project Master Template this project is built from.
`project_name`	Project Name	Data			Mandatory, Set once, List view	Name of this live project. Each tracker creates its own project.
`start_date`	Activation / Start Date	Date		Today	Set once	Drives every task due date. Used only when the live project is created.
`project`	Live Project	Link	Project Execution		Read-only, Unique, Standard filter	Created from the template on the first save and owned by this tracker.
`task`	Task	Link	Task Master		Mandatory, List view	Shown when `eval:doc.project_template || doc.project`.
`task_name`	Task Name	Data			Read-only	Fetched from `task.task_master`.
`remarks`	Remarks	Text			Mandatory, List view	Saving appends to this task's remark history; it never overwrites an earlier remark.
`pipeline_html`	Project Pipeline	HTML				
Entry Form						
`status`	Status	Select	WIP / Completed	WIP	Mandatory, List view, Standard filter	
`project_status`	Project Status	Select	(blank) / Active / Completed / On Hold / Cancelled			Set the overall status of the project. Leave blank to keep the progress-derived status.
`task_instance`	Task Instance	Link	Project Task Instance		Read-only	Resolved from the live project and the selected task.
Entry Details						
`entry_datetime`	Date & Time	Datetime			Read-only	
`concern_person`	Concern Person	Link	Employee		Read-only	
`entered_by`	Entered By	Link	User		Read-only	

Controller logic (`project_tracker.py`)

Hook / method	What it does
`before_insert` → `create_project`	Requires a template and a project name; refuses a template that is not submitted or is disabled; creates the tracker's own `Project Execution` with `start_date = start_date or today` using `ignore_permissions=True`, and stores its name in `project`. The project's `after_insert` then builds the task instances.
`validate` → `resolve_task_instance`	Maps the chosen Task Master to this project's own task instance (`project_execution = project AND task = task`) and throws when the task is not part of the project — the guard that prevents reporting on a foreign task.
`validate` → `validate_assignment`	Elevated roles pass. Otherwise the session user's active Employee must equal the task instance's `concern_person`, else `frappe.PermissionError`.
`validate` (remaining)	Stamps `entry_datetime` and `entered_by` when empty, resolves `concern_person` from the session user (falling back to the task's assignee), trims `remarks` and rejects an empty remark.
`on_update` → `apply_to_task_instance`	Runs on insert and on every edit: appends the remark to the resolved task instance with a fresh timestamp, then calls `Project Execution.update_progress(status=project_status)` so an explicit Project Status on the entry overrides the progress-derived one.
`on_trash`	Deletes the owned project: every task instance first (`force`, `delete_permanently`), then the `Project Execution`. Nothing else references them, so leaving them behind would strand a project no screen can reach or update.
`get_pipeline()` (whitelisted)	Delegates to `project_management.api.get_project_pipeline`.

Client behaviour (`project_tracker.js`)

- `project_template` link query is filtered to `docstatus = 1, disabled = 0`.
- `task` uses the server query `api.tracker_task_query`: before the live project exists options come from the template's rows, afterwards from the project's own task instances — so a task later removed from the template is still selectable on a running project. A Concern Person only ever sees their own assignments.
- `lock_project_fields` makes `project_template`, `project_name` and `start_date` read-only once the project exists, mirroring the `set_only_once` server rule, and clears the "created on save" hint that would otherwise misdescribe a saved entry.
- Selecting a template defaults `project_name` to the template name and renders the template preview; typing a name that already exists shows a warning that saving creates a separate project.
- After save the pipeline is rendered from `api.get_project_pipeline` — stages, status, due dates and the full remark trail.
- **Open Live Project** routes to the owned `Project Execution`. **Send Status to Admin Master** and **Send Manual Task Reminder** are rendered only after `permissions.can_manage_all_tasks` returns true, and both pass `[doctype, name]` so the message links back to the record being viewed.

Permissions

Role	Granted
System Manager	Read, Write, Create, Delete, Report, Export, Print, Email, Share
Admin Master	Read, Write, Create, Delete, Report, Export, Print, Email, Share
Concern Person	Read, Create, Report

> Concern Person has create + read + report but no `write`: an employee files entries and reads them back, while corrections to an existing entry stay with Admin Master / System Manager. Deleting a tracker is destructive by design — it removes the project it owns.

5.9 WhatsApp Reminder Log — outbound audit

Functional role. One row per outbound attempt of any kind, with the exact message body, the recipient, the provider message id, the outcome and the error or skip reason. It is both the delivery audit trail and the deduplication store that stops a retried job from messaging twice.

- Module: **Project Management for lands constructions etc**
- Naming: naming series `PM-WA-.#####`
- Rename allowed: no
- Change tracking (audit trail of field edits): off

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
`naming_series`	Series	Select	PM-WA-.#####	PM-WA-.#####	Set once	
`reminder_type`	Type	Select	Automatic Reminder / Manual Reminder / Status Summary		Mandatory, List view, Standard filter	
`project_execution`	Project	Link	Project Execution		Standard filter	
`task_instance`	Task Instance	Link	Project Task Instance			
`task_name`	Task	Data			List view	
`recipient_employee`	Recipient Employee	Link	Employee			
`recipient_name`	Recipient	Data			List view	
`recipient_number`	Recipient Number	Data				
`sent_by`	Triggered By	Link	User			
`attempt_datetime`	Attempt Time	Datetime			List view	
Message						
`message`	Message	Text				
Result						
`delivery_status`	Delivery Status	Select	Queued / Sent / Failed / Skipped	Queued	Mandatory, List view, Standard filter	
`idempotency_key`	Idempotency Key	Data			Unique	Deduplication key; a retry with the same key never sends twice (spec 8.3).
`provider_message_id`	Provider Message ID	Data				
`whatsapp_message`	WhatsApp Message	Link	WhatsApp Message			
`skip_reason`	Skip Reason	Small Text				
`error_response`	Error Response	Code	JSON			

Lifecycle (`whatsapp.py`)

1. `send_whatsapp_message(...)` looks up any existing log with the same `idempotency_key`. A key found in state `Sent`, `Skipped` or `Queued` returns that row and sends nothing; a `Failed` row is retried **in place**, reusing the row so the unique key stays unique.
2. Otherwise a new log row is created with `delivery_status = Queued`, the message body, recipient, triggering user and attempt time.
3. Reminders disabled globally → `Skipped`, reason "WhatsApp reminders are disabled in Project Management Settings", nothing sent. No recipient number → `Skipped`, reason "Recipient has no WhatsApp number configured".
4. `_dispatch` inserts an outgoing `WhatsApp Message` (frappe_whatsapp) with `reference_doctype = WhatsApp Reminder Log` and `reference_name = <log>`, then records `whatsapp_message`, `provider_message_id` and a delivery status taken **from the created document's own status** — a submitted API request is never reported as delivered. Recognised success states: `Success, sent, Sent, delivered, read, accepted`.
5. Any exception is logged to the Error Log and stored in `error_response` as JSON with `delivery_status = Failed`, which keeps the attempt retryable.

Idempotency key formats

Kind	Key	Effect
Automatic Reminder	`auto::<task instance>::<YYYY-MM-DD due slot>`	One send per task per due slot, however often the job is retried
Manual Reminder	`manual::<task instance>::<timestamp>`	Every manual press is a distinct, deliberate send
Status Summary	`status::<project>::<number>::<timestamp>`	Per recipient, per press

Permissions

Role	Granted
System Manager	Read, Write, Delete, Report, Export, Print, Email, Share
Admin Master	Read, Report, Export, Print

> Nobody has `create` and only System Manager has `write`: rows are written by the server with `ignore_permissions` and are an audit trail, not a data-entry screen. Admin Master reads, exports and prints them.

5.10 WhatsApp Status Recipient — child table of Settings

Functional role. The distribution list for **Send Status to Admin Master**: who receives project status summaries, on which number, and whether that entry is currently active.

- Module: **Project Management for lands constructions etc**
- Child table: rows exist only inside a parent document
- Rename allowed: yes
- Change tracking (audit trail of field edits): off

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
`employee`	Employee	Link	Employee		List view	
`recipient_name`	Recipient Name	Data			List view	Fetched from `employee.employee_name`.
`mobile_number`	WhatsApp Number	Data	Phone		Mandatory, List view	Fetched from `employee.cell_number`. Defaults to the Employee's Mobile; type a number here to override it.
`enabled`	Enabled	Check		1	List view	

Behaviour. `mobile_number` is mandatory and defaults to the Employee's `cell_number`, but stays editable so a recipient can be reached on a different number without editing the HR master. `Project Management Settings.get_status_recipients()` returns only rows with `enabled = 1` **and** a number, so clearing the checkbox pauses a recipient without losing the configuration.

Permissions

No permission rules of its own: access is inherited from the parent document.

5.11 Project Management Settings — single DocType

Functional role. The one place that controls outbound messaging: the master switch, the dispatch hour, the fallback interval, whether completed tasks may be chased manually, the public URL used in message links, and the status distribution list.

- Module: **Project Management for lands constructions etc**
- Single DocType: one global record, no list view
- Rename allowed: no
- Change tracking (audit trail of field edits): on

Fields

Fieldname	Label	Type	Options / Target	Default	Properties	Notes
Reminder Behaviour						
`enable_whatsapp_reminders`	Enable WhatsApp Reminders	Check		1		Master switch for all outbound WhatsApp messages.
`default_reminder_in_days`	Default Reminder in Days	Int		7		Fallback only. The interval is read from Project Master per task (spec 8.1).
`allow_manual_reminder_for_completed`	Allow Manual Reminders for Completed Tasks	Check		0		Spec 13 recommended default: disabled.
`reminder_send_hour`	Reminder Send Hour	Int		9		Hour of day (0-23) at which the daily reminder job dispatches messages.
`public_base_url`	Public Base URL	Data	URL			Public https:// address of this site, used for links in WhatsApp messages. WhatsApp only turns a URL into a tappable link when it has a real domain, so a localhost or 127.0.0.1 address arrives as plain text. Leave blank to use the site URL.
Admin Master Status Recipients						
`status_recipients`	Status Recipients	Table	WhatsApp Status Recipient			

Field semantics

Setting	Effect in code

`enable_whatsapp_reminders`	Checked first by `tasks.send_due_reminders` (returns immediately when off) and again by `whatsapp.send_whatsapp_message`, which logs a `Skipped` row instead of sending. Off means no outbound message of any kind, including manual ones.
`default_reminder_in_days`	Fallback value only. The live interval is always read from the task instance, which copied it from the template row — never hardcoded, never taken from Settings at send time.
`allow_manual_reminder_for_completed`	When off (the shipped default), `api.send_manual_task_reminder` refuses a Completed task and names it in the error.
`reminder_send_hour`	The single hour of the day in which dispatch happens. The scheduler runs hourly and returns unless `now().hour` matches. Tested for "configured", not truthiness, so `0` (midnight) is a valid hour.
`public_base_url`	Replaces the host of every outgoing link. WhatsApp only renders a URL as a tappable link when the host looks like a real domain, so `http://127.0.0.1:8007/...` or a `localhost` address arrives as plain text. Blank means the site's own URL is used.
`status_recipients`	Rows read by `get_status_recipients()`; an empty or fully disabled list makes **Send Status to Admin Master** throw "No Admin Master recipient configured".

Install-time defaults. `install.ensure_settings()` sets `default_reminder_in_days = 7`, `reminder_send_hour = 9` and `enable_whatsapp_reminders = 1` on a fresh install, and runs again after every migrate (`after_migrate`), filling the values only when `default_reminder_in_days` is empty.

Permissions

Role	Granted
System Manager	Read, Write, Create, Delete, Print, Email, Share
Admin Master	Read, Print

6. Reminder engine

6.1 Schedule and selection

`hooks.scheduler_events` registers `project_management.tasks.send_due_reminders` on `hourly_long`. Each run:

1. returns immediately if `enable_whatsapp_reminders` is off;
2. returns unless the current hour equals `reminder_send_hour`;
3. selects task instances where `current_status = WIP`, `enable_reminders = 1` and `next_reminder_due <= now`;
4. enqueues one background job per task on the `long` queue with `job_id = pm-reminder::<task>` and `deduplicate=True`, so a user's Save is never blocked and the same task cannot be queued twice concurrently.

6.2 Dispatch re-checks (`dispatch_reminder`)

The task is reloaded immediately before sending, because the world may have moved on:

Condition at dispatch	Action
Task is no longer WIP	`reminder_state = Skipped`, `next_reminder_due` cleared, a `Skipped` log row written with reason "Task is no longer WIP". Nothing sent.
Reminders turned off or interval ≤ 0	`reminder_state = Disabled`, due slot cleared, `Skipped` log row "Reminders disabled for this task".
Recomputed due slot is still in the future (a newer remark or send arrived)	The timer is restarted: `next_reminder_due` rewritten, `reminder_state = Scheduled`. Nothing sent.
Otherwise	Builds the body with `api.build_reminder_message(task)` and sends to `concern_person_mobile` as `Automatic Reminder`, with idempotency key `auto::<task>::<due date>`.
Result `Sent`	`mark_reminder_sent()` — stamps the send, sets state `Sent`, schedules the next slot.
Result `Skipped` / `Failed`	`reminder_state` set accordingly; a failed attempt stays retryable.

6.3 Interval arithmetic

The next due slot is `max(last_remark_on, last_reminder_sent_on) + reminder_in_days`, snapped to midnight. Two properties follow:

- **A remark resets the cycle.** Someone who updates a task is not chased again until the full interval has passed since that update.
- **The interval cannot drift.** Because the slot is a date, not a datetime, a fixed dispatch hour always finds the slot due on its own day; anchoring on a clock time would push each cycle one day later.

Anchoring only on the remark would freeze the slot and suppress every repeat, which is why the last successful send participates in the maximum.

7. WhatsApp messaging

Three outbound kinds, all logged in WhatsApp Reminder Log:

Kind	Trigger	Body
Automatic Reminder	Scheduler (`dispatch_reminder`)	Greeting with the concern person's name, request to update, project title, task name, latest remark and its date (or "No update recorded yet."), due date, closing line.
Manual Reminder	**Send Manual Task Reminder** (Admin Master / System Manager)	The same body plus the sender's optional **Additional Message** and a link to the document it was sent from. The typed note is appended, never substituted, so the project, task and latest remark always travel with it.
Status Summary	**Send Status to Admin Master**	Project title, project status, total / completed / WIP counts, the date, and a link to the document. No per-task breakdown by design — the recipient opens the link for detail.

`document_url(document)` builds the absolute desk URL for the record the message is about: it accepts `[doctype, name]` (as the client sends it), returns an empty string when the record is missing or the payload malformed, and rewrites the host with `public_base_url` when configured. `send_status_to_admin_master` falls back to the Project Execution's own URL when called without a document.

A manual reminder deliberately does **not** reset the automatic reminder timer.

8. Whitelisted API surface (`project_management/api.py`)

Method	Purpose	Guard
`task_instance_query`	Link query for task instances within one project	`validate_and_sanitize_search_inputs`; non-elevated users are filtered to their own assignments; no project filter returns an empty list
`tracker_task_query`	Task options for Project Tracker — project instances when the project exists, template rows before that	Same; a user with no active Employee record gets nothing
`get_template_projects(template)`	Up to 20 recent live projects of a template, for the tracker's duplicate-name hint	—
`get_project_pipeline(project)`	Ordered stages with status, assignee, due date and full remark history	`frappe.has_permission('Project Execution', doc=project, throw=True)`
`get_template_preview(template)`	Template rows shaped like a pipeline, with stages marked `Planned`, for a tracker before its project exists	`has_permission` on the template
`send_status_to_admin_master(project, document)`	Status summary to every enabled recipient; returns the body, an aggregate delivery status and the log rows	`require_reminder_permission()`; throws when no recipient is configured
`send_manual_task_reminder(task_instance, message, document)`	Manual reminder to the task's assigned concern person	`require_reminder_permission()`; refuses Completed tasks unless `allow_manual_reminder_for_completed`
`permissions.can_manage_all_tasks(user)`	Role probe used by the client to decide whether to render restricted buttons	Read-only; the server still re-checks every restricted action
`Project Execution.get_pipeline` / `Project Tracker.get_pipeline`	Document-bound pipeline for the form renderer	Document permissions
`Project Master Template.create_project_execution`	Create a live project from a submitted template	Re-checks `docstatus = 1` server-side

`build_reminder_message(task, note=None)` is the shared body builder (not whitelisted).

9. Hooks, migrations and installation

Hook	Value
`after_install` / `after_migrate`	`project_management.install.after_install`
`app_include_css`	`/assets/project_management/css/project_management.css`
`permission_query_conditions`	Project Task Instance, Project Tracker → `project_management.permissions.*`
`has_permission`	Project Task Instance, Project Tracker → `project_management.permissions.*`
`scheduler_events`	`hourly_long`: `project_management.tasks.send_due_reminders`

`after_install` creates the `Admin Master` and `Concern Person` roles (desk access) and seeds Settings.

Migration patches (`patches.txt`)

Patch	Stage	What it repairs
`restore_tracker_parent_task_fields`	pre-model-sync	Moves tracker task updates from the old child table to parent-level `task` / `remarks` / `status`. A tracker that carried several rows becomes several trackers, each attached to its own copy of the project rather than silently sharing one.
`split_shared_tracker_projects`	pre-model-sync	Gives every tracker its own Project Execution before the unique index on `Project Tracker.project` is added. The first tracker keeps the shared project; the others get their own, built from the same template.
`link_tracker_entries_to_remark_history`	post-model-sync	Claims legacy remark rows for their tracker entry and re-applies the entries, correcting task instances left without a remark row or with a stale status instead of duplicating rows.
`backfill_remark_status_change`	post-model-sync	Walks each task's history in order and derives `previous_status` / `status_changed` for rows written before those columns existed.
`backfill_tracker_template_fields`	post-model-sync	Derives `project_template`, `project_name` and `task` for entries that only stored `project` and `task_instance`, keeping them valid and editable.

Installation

```
cd $PATH_TO_YOUR_BENCH
bench get-app $URL_OF_THIS_REPO --branch version-16
bench install-app project_management
bench --site <site> migrate
```

10. Data lifecycle and deletion rules

Action	Consequence
Save a new Project Tracker	Creates one Project Execution and one Project Task Instance per template row; writes the first remark; refreshes project progress.
Change Task on an existing tracker and save	Appends a remark to the newly selected task. The previously selected task keeps everything written on it.
Re-save with the same remark and status	No history row is added; progress is recomputed harmlessly.
Set a task to Completed	`completed_on` stamped, reminder schedule cleared, `reminder_state = Disabled`, project counters and `progress_percent` updated; all tasks complete promotes the project to Completed.
Delete a Project Tracker	Deletes the project it owns, all its task instances and their remark history (permanently). Nothing else references them.
Create a project from a submitted template	A live project with no owning tracker; deleting it is a System Manager action.
Disable a Task Master / Type of Work / template	Blocks new configuration and new projects; existing projects and history are untouched.
Amend a submitted template	A new submitted version via `amended_from`; projects already created keep their own copies.

> Templates are frozen by submission and live projects hold independent copies, so editing or retiring a template never rewrites a project already under way.

11. Configuration checklist

1. Install the app and run `bench migrate`; confirm the `Admin Master` and `Concern Person` roles exist.
2. Assign roles to users, and make sure each Concern Person has an **Active Employee** record whose `user_id` is that user — row-level permissions and reminder recipients both depend on it.
3. Fill the Employee `cell_number` for everyone who should receive WhatsApp messages (overridable per task and per recipient row).
4. Configure **WhatsApp Account** in frappe_whatsapp (credentials and templates).
5. Open **Project Management Settings**: enable reminders, set the dispatch hour, set **Public Base URL** to the site's public `https://` address, and add the Admin Master status recipients.
6. Create Task Master and Type of Work Masters records.
7. Create and **submit** a Project Master Template with its task rows.
8. Verify the scheduler is running (`bench doctor`); reminders depend on `hourly_long`.

12. Troubleshooting

Symptom	Cause and fix
"Project Master Template … must be submitted before it can be used"	The template is still a draft, or disabled. Submit it and clear `disabled`.
"Task … is not part of project …"	The selected Task Master has no instance in this project — typically a task added to the template after the project was created. Projects hold their own copies by design.
"You are not the assigned Concern Person for this task"	A non-elevated user is reporting on someone else's task, or the user has no active Employee record mapped by `user_id`.
Task list in the tracker is empty	The user has no active Employee record, or no template row / task instance is assigned to them.
No reminders are sent	Check in order: `enable_whatsapp_reminders`; `reminder_send_hour` vs. the server clock; the scheduler is enabled; task is WIP with `enable_reminders` and a positive interval; `next_reminder_due` is in the past; the concern person has a mobile number.
Log row shows Skipped	Read `skip_reason`: reminders disabled globally, no recipient number, task no longer WIP, or reminders disabled on the task.
Log row shows Failed	Read `error_response` and the Error Log entry "Project Management WhatsApp send failed". The attempt stays retryable and reuses the same log row on the next run.
Links in messages are not tappable	`public_base_url` is blank or points at `localhost`/an IP. Set a real `https://` domain.
"No Admin Master recipient configured"	Settings has no recipient row with `enabled = 1` and a number.
Project status looks wrong	An explicit Project Status on a tracker entry overrides the derived status; Cancelled is never revived and On Hold is not demoted by partial progress. Clear the field to return to derived status.

13. Source map

Path	Contents
`project_management/hooks.py`	Permissions hooks, scheduler events, install hooks, CSS include
`project_management/api.py`	Whitelisted endpoints, link queries, message bodies, URL rewriting
`project_management/tasks.py`	Reminder scheduler and per-task dispatch
`project_management/whatsapp.py`	Outbound layer, logging, idempotency, provider outcome
`project_management/permissions.py`	Role probe, row-level conditions, action gate
`project_management/install.py`	Role creation and Settings defaults
`project_management/patches/`	Five migration patches
`…/project_masters/doctype/`	Task Master, Type of Work Masters, Project Master Template, Project Task Detail
`…/project_management_for_lands_constructions_etc/doctype/`	Project Execution, Project Task Instance, Task Remarks History, Project Tracker, WhatsApp Reminder Log, WhatsApp Status Recipient, Project Management Settings

`project_management/public/css/project_management.css`	Pipeline and remark-trail styling